

UNIVERSITÉ ALIOUNE DIOP DE BAMBEY

UFR SCIENCES APPLIQUÉES ET TECHNOLOGIES

MASTER 2 — DATA SCIENCE & GÉNIE LOGICIEL

RAPPORT DE PROJET BIG DATA

DIak-SN

**Plateforme Big Data pour la surveillance sanitaire et la gestion des
stocks de médicaments au Sénégal**

Projet : Big Data, Streaming et Machine Learning

Réalisé par :

Moussa GUEYE

Encadré par :

**Mr Ahmed Ben Sidy Bouya SEYE | Senior Big Data & AI
Engineer | Groupe Sonatel**

Année académique :

2025 – 2026

Technologies utilisées :

**Apache Kafka • Apache NiFi • Apache Spark • HBase •
Hadoop/HDFS • Hive • Airflow • Docker**

Bambey, Sénégal — 2026

1. Introduction

Le projet **Diak-SN** consiste à mettre en place une plateforme Big Data dédiée à la surveillance sanitaire au Sénégal. L'objectif principal est de collecter, traiter, stocker et analyser des données sanitaires en temps réel afin de faciliter la détection précoce des situations épidémiques et d'améliorer la gestion des stocks de médicaments.

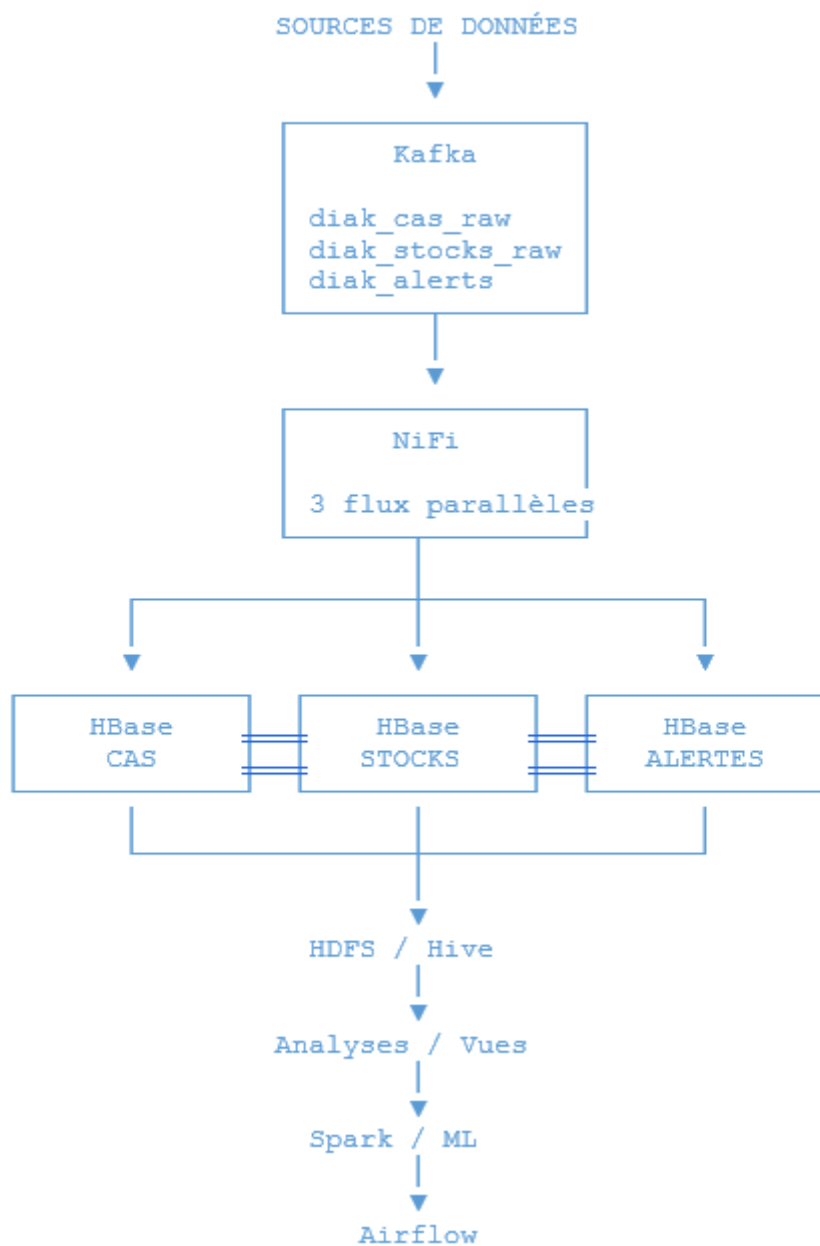
L'architecture retenue repose principalement sur :

- **Kafka** pour la transmission des données en temps réel ;
- **NiFi** pour l'ingestion et le routage des données ;
- **HBase** pour le stockage NoSQL temps réel ;
- **Hadoop HDFS** pour le stockage distribué ;
- **Hive** pour l'analyse des données ;
- **Spark** pour le traitement Big Data et le Machine Learning ;
- **Airflow** pour l'orchestration des traitements et le MLOps ;
- **PostgreSQL** comme base de données du Hive Metastore.

L'objectif de cette phase était d'abord de construire et de valider l'infrastructure avant de passer aux traitements analytiques et au Machine Learning.

2. Architecture générale réalisée

L'architecture actuellement mise en place peut être représentée comme suit :



2.1 Automatisation du démarrage avec

Un fichier important du projet est :

setup.sh

Ce script permet d'automatiser le démarrage de l'environnement.

Son objectif est d'éviter de démarrer manuellement chaque composant.

La logique générale est :

setup.sh



Cette automatisation constitue un élément important du projet car elle permet de préparer rapidement toute l'infrastructure nécessaire au traitement des données.

2.2 Génération des données avec

Le fichier :

kafka_producer.py

constitue le simulateur de données du projet.

Il permet de générer des données sanitaires simulées et de les publier dans Kafka.

Les principaux flux sont organisés autour de trois catégories :

2.2.1 Cas médicaux

Topic :

diak_cas_raw

Les données contiennent notamment :

```
cas_id
patient_id
district
pathologie
age_tranche
sexe
gravite
timestamp
```

Exemple conceptuel :

```
{
  "cas_id": "CAS001",
  "patient_id": "PAT001",
  "district": "Dakar",
  "pathologie": "Paludisme",
  "age_tranche": "15-24",
  "sexe": "M",
  "gravite": "moderee",
  "timestamp": "2026-09-13T08:00:00"
}
```

2.2.2 Stocks de médicaments

Topic :

diak_stocks_raw

Les informations transmises sont notamment :

district
medicament
stock_actuel
stock_mini
timestamp

2.2.3 Alertes

Topic :

diak_alerts

Ce flux permet de transmettre les alertes générées par le système.

3. Mise en place de l'environnement Docker

L'ensemble de l'infrastructure a été déployé à l'aide de Docker Compose.

Les principaux conteneurs utilisés sont :

Conteneur	Technologie	Fonction
zookeeper	Confluent Zookeeper 7.2.1	Coordination de Kafka
kafka	Confluent Kafka 7.2.1	Streaming des données
nifi	Apache NiFi 1.23.2	Ingestion et routage
hbase	HBase	Stockage NoSQL temps réel
namenode	Hadoop NameNode	Gestion du système HDFS
datanode	Hadoop DataNode	Stockage physique HDFS
postgres	PostgreSQL 13	Base du Hive Metastore
hive-metastore	Hive 2.3.2	Métadonnées Hive
hive-server	Hive 2.3.2	Exécution des requêtes Hive
spark-master	Apache Spark 3.5.0	Traitement distribué
spark-worker	Apache Spark 3.5.0	Exécution des tâches Spark
airflow	Apache Airflow 2.7.3	Orchestration/MLOps

Les composants Kafka, Zookeeper, NiFi, HBase et PostgreSQL ont été démarrés et vérifiés.

4. Mise en place de Kafka

Kafka constitue la couche de transmission des données en temps réel.

Les principaux topics utilisés dans le projet sont :

```
diak_cas_raw  
diak_stocks_raw  
diak_alerts
```

Ces topics correspondent respectivement aux :

- données de cas médicaux ;
- données de stocks de médicaments ;
- alertes épidémiologiques.

Kafka est exécuté dans un conteneur :

```
confluentinc/cp-kafka:7.2.1
```

avec le port :

```
9092
```

Zookeeper assure la coordination du cluster Kafka.

5. Mise en place de NiFi

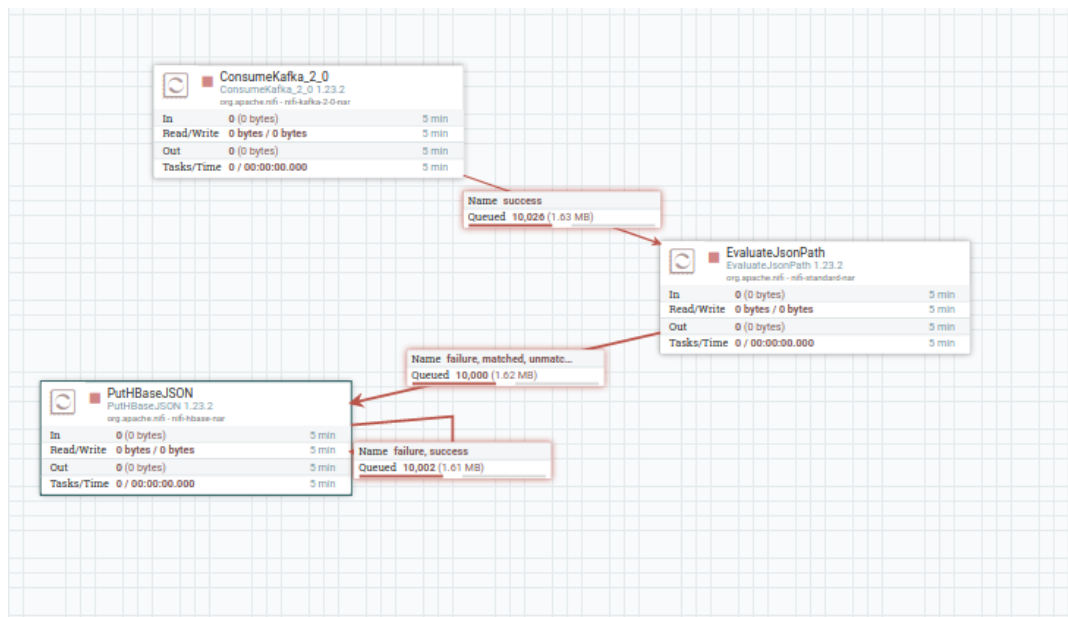
Apache NiFi a été configuré pour recevoir les données provenant de Kafka et les acheminer vers HBase.

NiFi est accessible sur :

```
http://localhost:8081
```

Trois templates NiFi ont été créés puis importés dans le Process Group.

L'architecture comporte trois flux parallèles.



5.2 Flux des stocks

Le deuxième flux traite les stocks de médicaments.

```

ConsumeKafka_2_0
    ↓
EvaluateJsonPath
    ↓
PutHBaseJSON
    ↓
diak:stocks_medicaments
  
```

Le topic Kafka utilisé est :

diak_stocks_raw

Le Group ID est :

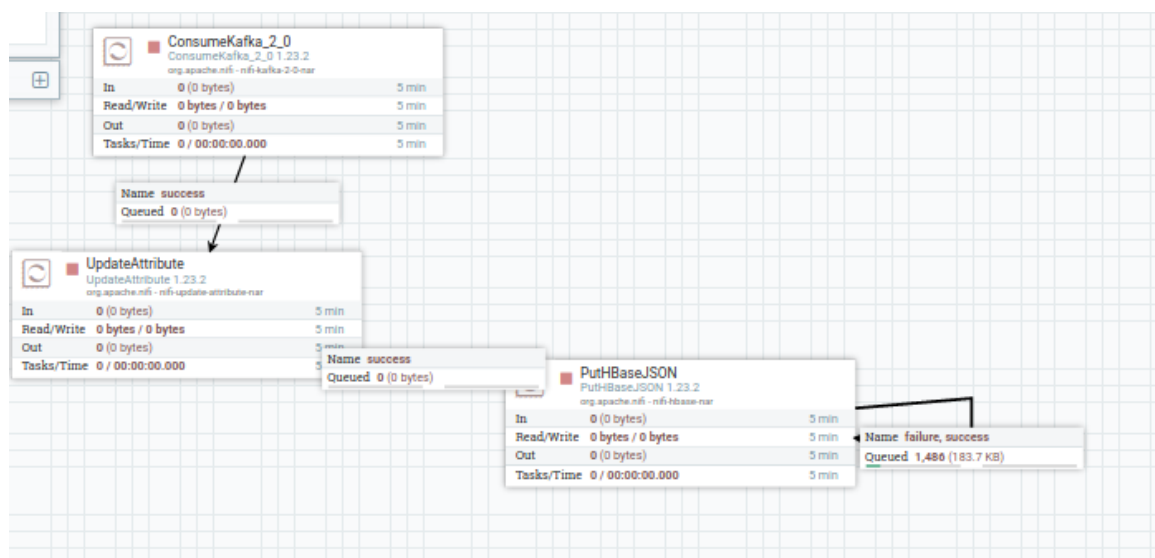
nifi-stocks

Les propriétés extraites avec EvaluateJsonPath sont :

Attribut	JSONPath
district	\$.district
medicament	\$.medicament
stock_actuel	\$.stock_actuel
stock_mini	\$.stock_mini
timestamp	\$.timestamp

La table HBase cible est :

diak:stocks_medicaments



5.3 Flux des cas médicaux

Le troisième flux traite les cas médicaux.

```
ConsumeKafka_2_0
↓
EvaluateJsonPath
↓
PutHBaseJSON
↓
diak:cas_temps_reel
```

Le topic utilisé est :

diak_cas_raw

Le Group ID est :

nifi-cas

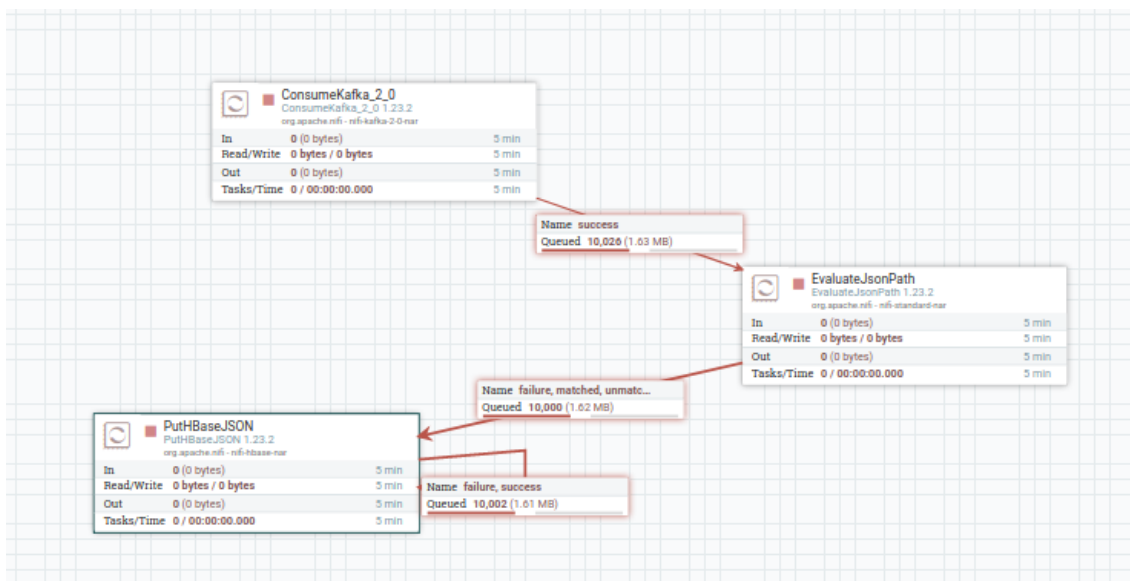
Les données extraites sont :

Attribut	JSONPath
cas_id	\$.cas_id
patient_id	\$.patient_id
district	\$.district

Attribut	JSONPath
pathologie	\$.pathologie
age_tranche	\$.age_tranche
sexe	\$.sexe
gravite	\$.gravite
timestamp	\$.timestamp

La table HBase cible est :

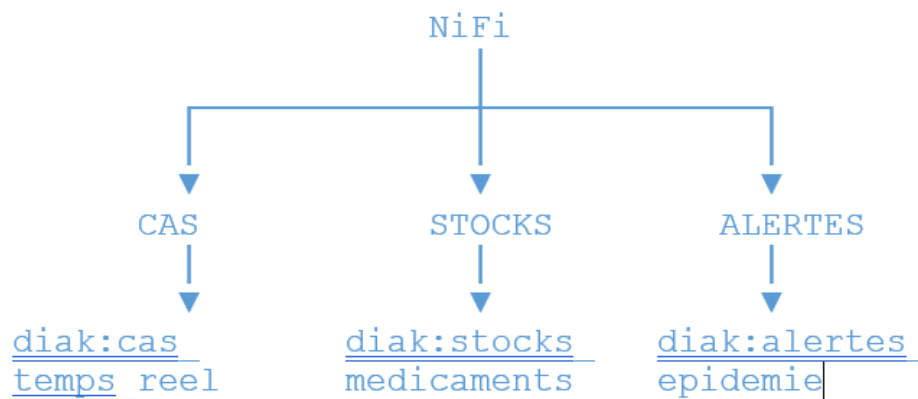
diak:cas_temps_reel



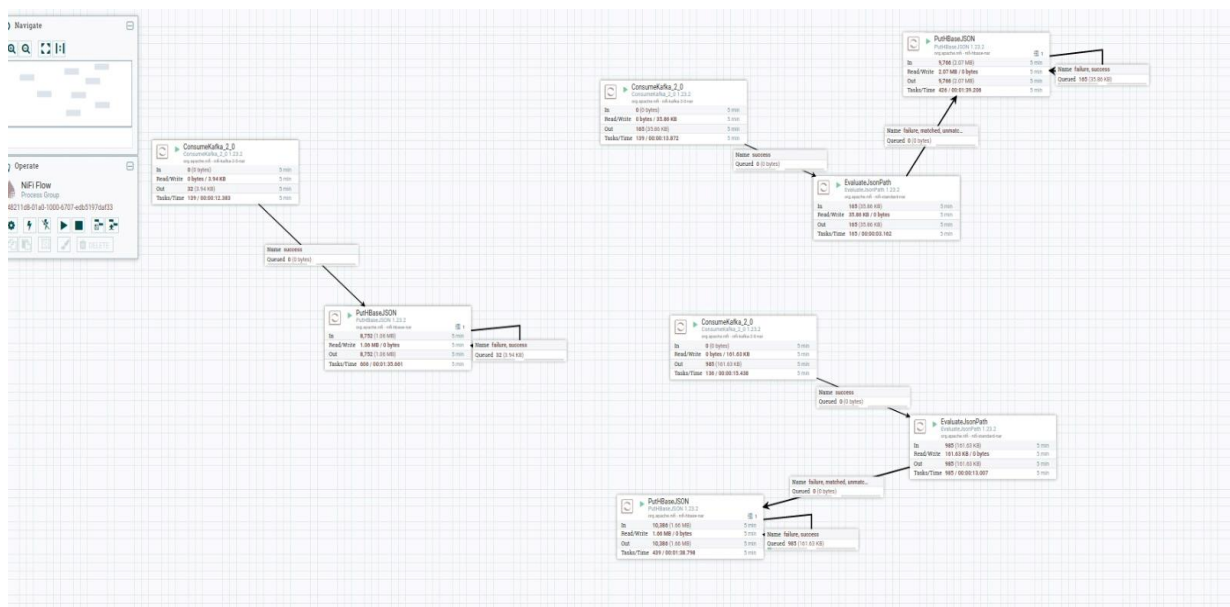
6. Résultat important : écriture simultanée dans HBase

L'un des objectifs importants de cette étape était de vérifier que NiFi pouvait alimenter simultanément les trois tables HBase.

Les trois flux ont finalement été configurés et **NiFi écrit maintenant correctement dans les trois tables** :



Voici le flow file complete de nifi



En fin le resultat de l'ecriture dans les tables Hbase :

```

(venv) moussa@moussa-VirtualBox:~/uadb-m2-diak-sn$ sudo docker exec -it hbase hbase shell
[sudo] Mot de passe de moussa :
2026-09-12 08:02:28,009 WARN [main] util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
HBase Shell
Use "help" to get list of supported commands.
Use "exit" to quit this interactive shell.
For Reference, please visit: http://hbase.apache.org/2.0/book.html#shell
Version 2.1.3, rda5ec9e4c06c537213883cca8f3cc9a7c19daf67, Mon Feb 11 15:45:33 CST 2019
Took 0.0319 seconds
hbase(main):001:0> scan 'diak:alertes_epidemie'
ROW                                COLUMN+CELL
district                           column=meta:alerte_epidemie, timestamp=1789200078862, value=NORMAL
district                           column=meta:district, timestamp=1789200078862, value=DAKAR
district                           column=meta:nb_cas_7j, timestamp=1789200078862, value=8
district                           column=meta:pathologie, timestamp=1789200078862, value=PALUDISME
district                           column=meta>window_end, timestamp=1789200078862, value=2026-09-15T00:00:00.000Z
district                           column=meta>window_start, timestamp=1789200078862, value=2026-09-08T00:00:00.000Z
1 row(s)
Took 2.8464 seconds
  
```

```

hbase(main):002:0> scan 'diak:cas_temps_reel'
COLUMN+CELL
ROW
cas id      column=meta:age_tranche, timestamp=1789202108124, value=0-5
cas id      column=meta:cas_id, timestamp=1789202108124, value=07dcaa8-01c9-4bbe-90ac-8432cc231993
cas id      column=meta:district, timestamp=1789202108124, value=THIES
cas id      column=meta:gravite, timestamp=1789202108124, value=GRAVE
cas id      column=meta:pathologie, timestamp=1789202108124, value=DIARRHEE
cas id      column=meta:patient_id, timestamp=1789202108124, value=PAT_DA61C10FCA
cas id      column=meta:sexe, timestamp=1789202108124, value=F
cas id      column=meta:timestamp, timestamp=1789202108124, value=2026-09-12T07:57:54Z
1 row(s)
Took 1.2269 seconds

```

```

1 row(s)
Took 1.2269 seconds
hbase(main):003:0> scan 'diak:stocks_medicaments'
COLUMN+CELL
ROW
DAKAR      column=meta:medicament, timestamp=1789200078542, value=SR0
DAKAR      column=meta:stock_actuel, timestamp=1789200078542, value=869
DAKAR      column=meta:stock_mini, timestamp=1789200078542, value=160
DAKAR      column=meta:timestamp, timestamp=1789200078542, value=2026-09-12T07:57:45Z
KAOLACK    column=meta:medicament, timestamp=1789200078542, value=COARTEM
KAOLACK    column=meta:stock_actuel, timestamp=1789200078542, value=217
KAOLACK    column=meta:stock_mini, timestamp=1789200078542, value=100
KAOLACK    column=meta:timestamp, timestamp=1789200078542, value=2026-09-12T07:57:48Z
LOUGA     column=meta:medicament, timestamp=1789200078542, value=SR0
LOUGA     column=meta:stock_actuel, timestamp=1789200078542, value=94
LOUGA     column=meta:stock_mini, timestamp=1789200078542, value=160
LOUGA     column=meta:timestamp, timestamp=1789200078542, value=2026-09-12T07:58:40Z
SAINT LOUIS column=meta:medicament, timestamp=1789200078542, value=AML00IPINE
SAINT LOUIS column=meta:stock_actuel, timestamp=1789200078542, value=180
SAINT LOUIS column=meta:stock_mini, timestamp=1789200078542, value=80
SAINT LOUIS column=meta:timestamp, timestamp=1789200078542, value=2026-09-12T07:57:33Z
TAMBACOUNDA column=meta:medicament, timestamp=1789200078542, value=SR0
TAMBACOUNDA column=meta:stock_actuel, timestamp=1789200078542, value=1052
TAMBACOUNDA column=meta:stock_mini, timestamp=1789200078542, value=160
TAMBACOUNDA column=meta:timestamp, timestamp=1789200078542, value=2026-09-12T07:58:03Z
THIES     column=meta:medicament, timestamp=1789200078542, value=SR0
THIES     column=meta:stock_actuel, timestamp=1789200078542, value=892
THIES     column=meta:stock_mini, timestamp=1789200078542, value=160
THIES     column=meta:timestamp, timestamp=1789200078542, value=2026-09-12T07:58:18Z
ZIGUINCHOR column=meta:medicament, timestamp=1789200078542, value=PARACETAMOL
ZIGUINCHOR column=meta:stock_actuel, timestamp=1789200078542, value=2013
ZIGUINCHOR column=meta:stock_mini, timestamp=1789200078542, value=400
ZIGUINCHOR column=meta:timestamp, timestamp=1789200078542, value=2026-09-12T07:58:12Z
-          column=meta:district, timestamp=1789202163478, value=ZIGUINCHOR
-          column=meta:medicament, timestamp=1789202163478, value=PARACETAMOL
-          column=meta:stock_actuel, timestamp=1789202163478, value=2013
-          column=meta:stock_mini, timestamp=1789202163478, value=400
-          column=meta:timestamp, timestamp=1789202163478, value=2026-09-12T07:58:12Z
8 row(s)
Took 2.4771 seconds
hbase(main):004:0>

```

Cette étape valide donc le fonctionnement de la chaîne :

Kafka → NiFi → HBase

pour les trois catégories de données.

7. Réinitialisation du stockage HBase

Une ancienne configuration HBase contenait des métadonnées et des données de test.

Le volume utilisé par HBase était :

```
~/uadb-m2-diak-sn/hbase-data
```

Afin de repartir sur une base propre, les anciennes données du volume ont été supprimées avec :

```
sudo rm -rf ~/uadb-m2-diak-sn/hbase-data/*
```

Le volume a ainsi été réinitialisé sans supprimer le conteneur HBase lui-même.

Cette opération a permis de repartir avec des tables HBase propres.

8. Mise en place de Hadoop HDFS

Une couche Hadoop HDFS a ensuite été ajoutée pour préparer le stockage distribué nécessaire à Hive.

Deux composants principaux ont été déployés :

```
namenode  
datanode
```

Le NameNode est accessible sur le port :

```
8020
```

Le DataNode est également opérationnel.

La commande suivante a permis de vérifier l'état de HDFS :

```
sudo docker exec namenode hdfs dfsadmin -report
```

Le résultat obtenu indique :

```
Configured Capacity: 38.63 GB  
Present Capacity: 5.62 GB  
DFS Remaining: 5.62 GB  
DFS Used: 44 KB  
Missing blocks: 0  
Corrupt replicas: 0  
Live datanodes: 1
```

Cela confirme que le DataNode est bien connecté au NameNode.

9. Vérification de HDFS depuis Hive

Un problème de configuration a été rencontré avec Hive.

Initialement, Hive essayait de communiquer avec :

```
hdfs://hive-server:8020
```

alors que le véritable NameNode est :

```
namenode:8020
```

Le fichier :

```
/opt/hadoop-2.7.4/etc/hadoop/core-site.xml
```

a donc été corrigé.

La configuration correcte est :

```
<property>
  <name>fs.defaultFS</name>
  <value>hdfs://namenode:8020</value>
</property>
```

Après correction, la commande :

```
sudo docker exec hive-server bash -c \
'$HADOOP_HOME/bin/hdfs getconf -confKey fs.defaultFS'
```

retourne :

```
hdfs://namenode:8020
```

La communication avec HDFS a ensuite été testée :

```
sudo docker exec hive-server bash -c \
'$HADOOP_HOME/bin/hdfs dfs -ls /'
```

Résultat :

```
Found 1 items
drwxr-xr-x ... /test-diak
```

La même vérification depuis `hive-metastore` a également fonctionné.

Cette étape confirme que :

```
Hive Server → HDFS NameNode
```

fonctionne correctement.

10. Vérification du DataNode

L'exécution de :

```
sudo docker exec namenode hdfs dfsadmin -report
```

a confirmé la présence d'un DataNode actif.

Résultat observé :

```
Live datanodes (1):
```

```
Name: ...:50010
```

```
Hostname: datanode
Decommission Status: Normal
```

Le DataNode dispose d'environ :

38,63 GB

de capacité configurée.

Cette vérification montre que l'infrastructure HDFS est opérationnelle.

11. Mise en place de PostgreSQL pour Hive Metastore

PostgreSQL est utilisé comme base de métadonnées pour Hive.

Configuration :

```
Base de données : metastore
Utilisateur      : hive
Mot de passe     : hive
Port             : 5432
```

Le conteneur utilisé est :

postgres:13

Le serveur PostgreSQL a correctement démarré :

```
database system is ready to accept connections
```

Hive Metastore utilise cette base pour stocker les informations concernant les bases, tables, partitions et autres métadonnées Hive.

12. Préparation de Hive

Le fichier `hive_setup.sql` a été préparé afin de créer la couche analytique du projet.

La base prévue est :

```
CREATE DATABASE IF NOT EXISTS diak_sn;
```

Deux tables principales sont prévues.

12.1 Table des cas journaliers

`cas_journaliers`

Cette table doit contenir :

- `cas_id` sécurisé ;
- `district` ;
- `pathologie` ;
- `tranche d'âge` ;
- `sexe` ;
- `gravité` ;
- `timestamp d'ingestion` ;
- `date d'observation` ;
- `partition par pathologie`.

Le format de stockage prévu est :

ORC

avec compression :

SNAPPY

12.2 Table des stocks

La deuxième table est :

`stocks_medicaments`

Elle contient :

- `district` ;
- `médicament` ;
- `stock actuel` ;
- `stock minimum` ;
- `timestamp d'ingestion`.

Elle sera également stockée au format ORC.

13. Vues analytiques prévues

Deux vues analytiques ont été définies.

Vue des alertes épidémiologiques

`vue_alertes_districts`

Cette vue permettra de calculer le nombre de cas sur les sept derniers jours par :

`district + pathologie`

Elle permettra également de déterminer automatiquement un statut :

CRITIQUE

ALERTE

NORMAL

selon le nombre de cas.

Vue des ruptures de stock

La deuxième vue est :

`vue_ruptures_stock`

Elle permettra d'identifier les médicaments dont le stock est proche ou inférieur au stock minimum.

Le pourcentage de stock sera calculé afin de faciliter l'identification des ruptures imminentes.

14. Spark

Apache Spark 3.5.0 a été intégré dans l'infrastructure.

Deux conteneurs sont prévus :

`spark-master`

`spark-worker`

Le Spark Master utilise :

`7077`

pour la communication avec les workers.

L'interface Web du Master est accessible sur :

`8080`

Spark constitue la couche qui sera utilisée pour :

- le traitement des données ;

- l'analyse des séries temporelles ;
- la préparation des données Machine Learning ;
- la prédiction des pics de consultations ;
- le calcul des indicateurs sanitaires.

15. Airflow et MLOps

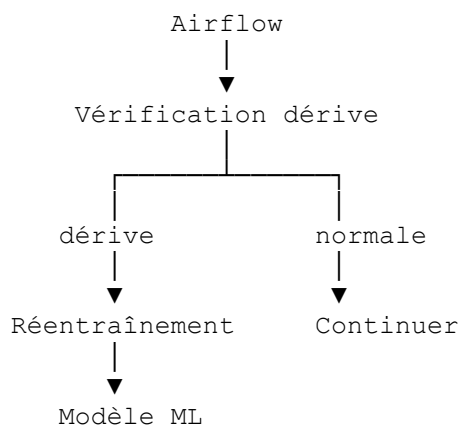
Airflow 2.7.3 a été prévu pour l'orchestration des traitements.

Le DAG `diak_sn_dag.py` doit notamment utiliser :

`BranchPythonOperator`

pour décider automatiquement si un réentraînement du modèle est nécessaire.

Le principe prévu est :



Les recommandations produites doivent ensuite être écrites dans HBase afin de permettre un accès temps réel.

16. Gestion des ressources Docker

Compte tenu des ressources limitées de la machine virtuelle, une attention particulière a été portée à la consommation mémoire.

Les services indispensables au fonctionnement actuel sont :

```

zookeeper
kafka
nifi
hbase
namenode
datanode
postgres
hive-metastore
  
```

hive-server

Les services Spark et Airflow peuvent être démarrés lorsque leurs traitements sont nécessaires.

Cette organisation permet de limiter la consommation de RAM lorsque toutes les couches ne sont pas utilisées simultanément.

17. État actuel du projet

À ce stade, les éléments suivants sont **réalisés et vérifiés** :

Composant	État
Docker	✓ Opérationnel
Zookeeper	✓ Opérationnel
Kafka	✓ Opérationnel
Topics Kafka	✓ Configurés
NiFi	✓ Opérationnel
Templates NiFi	✓ Importés
Flux Cas	✓ Fonctionnel
Flux Stocks	✓ Fonctionnel
Flux Alertes	✓ Fonctionnel
Écriture NiFi → HBase	✓ Fonctionnelle
Tables HBase	✓ Créées
HBase simultané sur 3 tables	✓ Vérifié
HDFS NameNode	✓ Opérationnel
HDFS DataNode	✓ Opérationnel
Communication Hive → HDFS	✓ Vérifiée
PostgreSQL	✓ Opérationnel

18. Problèmes rencontrés et solutions

18.1 Anciennes métadonnées HBase

Le volume HBase conservait d'anciennes données et métadonnées.

Solution

Le contenu du volume :

```
hbase-data
```

a été nettoyé afin de repartir avec une installation propre.

18.2 Échec initial de communication avec HDFS

Hive essayait initialement de contacter :

```
hive-server:8020
```

alors que le NameNode réel était :

```
namenode:8020
```

Solution

Le fichier `core-site.xml` a été corrigé pour utiliser :

```
hdfs://namenode:8020
```

Après correction, les commandes HDFS fonctionnent depuis Hive Server et Hive Metastore.

18.3 Mauvais chemin `HADOOP_CONF_DIR`

La configuration initiale indiquait :

```
/opt/hadoop/etc/hadoop
```

alors que Hadoop 2.7.4 utilise :

```
/opt/hadoop-2.7.4/etc/hadoop
```

Solution

La variable a été corrigée :

```
HADOOP_HOME=/opt/hadoop-2.7.4
HADOOP_CONF_DIR=/opt/hadoop-2.7.4/etc/hadoop
```

19. Résultats obtenus

Le résultat le plus important de cette première phase est la validation de la chaîne temps réel :

```
Kafka
  ↓
NiFi
  ↓
HBase
```

avec trois flux fonctionnant simultanément :

```
Cas médicaux  → diak:cas_temps_reel
Stocks        → diak:stocks_medicaments
Alertes       → diak:alertes_epidemie
```

L'infrastructure HDFS a également été validée :

```
Hive Server
  ↓
HDFS
  ↓
NameNode
  ↓
DataNode
```

La commande `hdfs dfs -ls /` permet désormais d'accéder correctement à HDFS depuis les conteneurs Hive.

21. Conclusion

La première phase du projet Diak-SN a permis de mettre en place et de valider une grande partie de l'infrastructure Big Data.

La chaîne **Kafka** → **NiFi** → **HBase** est maintenant opérationnelle. Les trois flux correspondant aux cas médicaux, aux stocks de médicaments et aux alertes épidémiologiques fonctionnent simultanément et permettent l'écriture dans les trois tables HBase dédiées.

Par ailleurs, l'infrastructure Hadoop a été mise en place avec un NameNode et un DataNode fonctionnels. Les problèmes de configuration rencontrés au niveau de Hive et de HDFS ont été identifiés puis corrigés, notamment la configuration de `fs.defaultFS` et de

`HADOOP_CONF_DIR`.

Le projet dispose donc maintenant d'une base technique solide permettant de poursuivre vers la couche analytique Hive, les traitements Spark, le Machine Learning, l'orchestration Airflow et la réalisation du dashboard final.

La prochaine priorité est de **finaliser Hive et valider les tables et vues analytiques**, puis d'enchaîner sur Spark et le modèle de prédiction.